FLEETFLY: FUEL INTELLIGENCE & PREDICTIVE EFFICIENCY

A Practical Integration of MongoDB, PyMongo, and Machine Learning in Fleet Management

Degree Program: B.Tech Computer Science & Engineering | **Course:** DBMS & ML Laboratory
Live Base Project: https://www.thefleetfly.xyz/ | **Academic Session:** 2026

Abstract: Fleet management applications generate substantial telemetry logs, with fuel representing up to 40% of fleet operating costs. This project develops **Fuel Intelligence**, an end-to-end predictive feature integrated into FleetFly. The system employs **MongoDB** as a polyglot document store, uses **PyMongo** for CRUD, indexing, and multi-stage aggregation pipelines, and implements supervised machine learning via **Linear Regression** (continuous MPG) and a **Decision Tree Classifier** (efficiency tiers). On the held-out test split of the UCI Auto MPG benchmark dataset, the Linear Regression model achieves an **R^2 of 0.7861** (MAE 2.6732 MPG), while the Decision Tree achieves an **Accuracy of 88.75%**, **Precision of 0.8974**, **Recall of 0.8750**, and an **F1 Score of 0.8861**. All code, schemas, and verification pipelines are designed for complete academic reproducibility.

1. Project Overview & Problem Statement

Commercial fleet operations require accurate asset monitoring, maintenance coordination, and operating expense management. While FleetFly currently tracks assets and schedules maintenance, fuel logs lacked predictive insight. Fleet managers need proactive estimation of expected fuel economy derived from vehicle mechanical specifications (cylinders, displacement, horsepower, and weight) before deployment.

The objective of this project is to integrate a lightweight Python data science service (ml_service/) with MongoDB using **PyMongo**, train supervised machine learning models, and present interactive analytics seamlessly within the existing React + Tailwind CSS dashboard.

2. System Architecture & Polyglot Persistence

The architecture combines an existing Node.js/Express backend with a dedicated Python PyMongo ML pipeline sharing the same MongoDB database (fleetdb):

Layer	Technology	Role & Key Responsibilities
Client (Frontend)	React 18, Vite, Tailwind CSS	Fuel Intelligence dashboard, interactive sliders, live predictions, metric cards
API Server	Node.js, Express, Mongoose	REST endpoints for fuels, vehicles, predictions, and summary statistics
ML Service	Python 3.14, PyMongo, scikit-learn	Data cleaning, model training, evaluation metrics, PyMongo CRUD & aggregation
Database Layer	MongoDB Atlas (fleetdb)	Collections: vehicles, fuellogs, prediction_logs, ml_model_metrics, sandbox

3. Benchmark Dataset & Data Preprocessing

Because the demonstration database currently contains 3 vehicles and zero historical consecutive fuel records, synthesizing fake records to train a model would violate academic integrity. In accordance with project guidelines, models are trained on the verified **UCI Auto MPG Benchmark Dataset** (398 instances).

Preprocessing Pipeline:

- **Missing Value Imputation:** 6 missing horsepower values ('?') were imputed via median (93.5 HP).
- **Target Definition:** Continuous target is MPG. Classification target is High Efficiency (1 if MPG >= median 23.00, 0 otherwise; class distribution: 201 vs 197).
- **Stratified Train-Test Split:** 80% training (318 samples) and 20% held-out test (80 samples) with fixed random seed (42).
- **Feature Standardization:** StandardScaler fitted strictly on training data and serialized via joblib.

4. PyMongo Implementation & Representative Queries

The Python implementation in ml_service/src/mongo_client.py demonstrates all required database operations:

- **Connection:** MongoClient(os.getenv('MONGODB_URI')) connecting to database 'fleetdb'.
- **Read:** Queries existing vehicles and fuel logs using projections and chronological sorting.
- **Create / Insert:** Persists evaluation records to ml_model_metrics and vehicle predictions to prediction_logs.
- **Filtered Query & Update:** Queries predictions (predicted_mpg >= 25.0) and updates audit notes.
- **Safe Deletion:** Tests delete_one() strictly inside an isolated sandbox collection (ml_demo_sandbox), never touching fleet data.
- **Multi-Stage Aggregation Pipeline:** Groups fuel logs by vehicle, calculating total liters, total cost, distance traveled, and km/L:

```
pipeline = [
    { '$match': { 'liters': { '$gt': 0 }, 'cost': { '$gt': 0 } } },
    { '$group': { '_id': '$vehicle', 'totalLiters': { '$sum': '$liters' }, 'totalCost': { '$sum': '$c
ost' },
                  'minOdo': { '$min': '$odometer' }, 'maxOdo': { '$max': '$odometer' } } },
    { '$lookup': { 'from': 'vehicles', 'localField': '_id', 'foreignField': '_id', 'as': 'v' } },
    { '$project': { 'plate': '$v.plateNumber', 'liters': '$totalLiters', 'cost': '$totalCost',
                    'distance': { '$subtract': ['$maxOdo', '$minOdo'] } } }
]
```

5. Machine Learning Models & Experimental Results

All metrics below represent **actual experimental results** computed on the 20% held-out test partition (80 samples) and recorded in ml_service/results/metrics.json.

A. Linear Regression (Continuous Fuel Economy in MPG)

Metric	Test Result	Academic Interpretation
R^2 Score	0.7861	The model explains 78.61% of fuel economy variance
Mean Absolute Error (MAE)	2.6732 MPG	Average prediction error across held-out vehicles
Root Mean Squared Error (RMSE)	3.5554 MPG	Standard deviation of prediction residuals
Model Intercept	23.4811	Baseline economy for an average-specification vehicle

B. Decision Tree Classifier (Efficiency Tier Categorization, max_depth=4)

Metric	Test Result	Target / Positive Class Specification
Accuracy	0.8875 (88.75%)	Overall classification correctness across test set
Precision	0.8974 (89.74%)	Positive Class: High Efficiency (MPG >= 23.00)
Recall (Sensitivity)	0.8750 (87.50%)	Fraction of true high-efficiency vehicles identified
F1 Score	0.8861 (88.61%)	Harmonic mean of precision and recall
Confusion Matrix	TN: 36, FP: 4 / FN: 5, TP: 35	36 true standard, 35 true high efficiency identified

6. Full-Stack UI Enhancements in FleetFly

The feature was integrated directly into existing FleetFly pages without breaking existing functionality:

- **Fuel Logs Page (/dashboard/fuels):** Upgraded from a coming-soon placeholder to a full dashboard with 4 summary metric cards, an interactive ML Prediction Panel (sliders for cylinders, displacement, horsepower, weight), real-time inference, prediction history from MongoDB, demo dataset loader, and fuel log entry modal.
- **Reports Page (/dashboard/reports):** Integrated the ML Model Evaluation section displaying actual test metrics for both models, confusion matrix, and dataset attribution.
- **Maintenance Page (/dashboard/maintenance):** Added a compact navigation link pointing to Fuel Intelligence without altering maintenance schedules.

7. Reproducible Execution Instructions

To execute and verify the complete project on any standard system:

```
# 1. Install dependencies
pip install -r ml_service/requirements.txt
npm install --prefix backend && npm install --prefix frontend

# 2. Execute PyMongo operations and ML training pipeline
python -m ml_service.src.run_all

# 3. Launch FleetFly application
npm start --prefix backend      # Starts API on Port 5001
npm run dev --prefix frontend   # Starts Vite UI on Port 5173
```

8. Conclusion

The FleetFly Fuel Intelligence feature successfully integrates MongoDB document-oriented operations via PyMongo with supervised machine learning. The project achieves high predictive accuracy (R^2 = 0.7861, F1 = 0.8861) while adhering to professional software engineering standards, polyglot database design, and reproducible academic guidelines.